

DOCUMENT INTERNE

MANUEL TECHNIQUE
LES ROUAGES DU PROJET, PAS À PAS

GEOCRAS

Installer, comprendre et modifier le projet

— OBJET

Maîtrise du code

— DESTINATAIRE

MFEE Georges
usage personnel

À QUI S'ADRESSE CE MANUEL

Ce document est écrit pour vous, et pour personne d'autre. Là où le cahier des charges explique le produit à quelqu'un qui l'évalue, celui-ci explique la mécanique à celui qui doit la maintenir. Rien n'y est vendu ; tout y est démonté.

L'ordre suit celui d'un apprentissage réel plutôt que celui d'une table des matières logique. On commence par installer et faire tourner, parce qu'on comprend mal une architecture qu'on n'a jamais vue démarrer. On passe ensuite aux bibliothèques une par une, puis à la façon dont les morceaux se tiennent, puis aux algorithmes. Les pièges connus sont regroupés à la fin, mais ils sont aussi signalés au moment où ils peuvent frapper.

Une promesse et une réserve. La promesse : tout ce qui est écrit ici a été relu dans le code, pas reconstitué de mémoire — les versions, les constantes, les noms de fonctions et les chemins de fichiers sont ceux du dépôt. La réserve : le code bouge, ce document non. Les numéros de ligne sont donc absents partout, et les noms de fonctions présents partout : un nom se retrouve avec une recherche, un numéro de ligne mène à la mauvaise ligne dès le premier commit.

SOMMAIRE

À QUI S'ADRESSE CE MANUEL	2
1 VUE D'ENSEMBLE EN CINQ MINUTES	6
1.1 Ce que le système fait, mécaniquement	6
1.2 Les trois paquets et la règle qui les gouverne	6
1.3 Les quatre couches	7
2 INSTALLER LE PROJET, PAS À PAS	8
2.1 La vue d'ensemble de l'installation	8
2.2 Étape 0 — Ce qu'il faut sur la machine	8
2.3 Étape 1 — Cloner et installer	9
2.4 Étape 2 — Compiler le paquet partagé	9
2.5 Étape 3 — La base de données	9
2.6 Étape 4 — Lancer le serveur	10
2.7 Étape 5 — Lancer le mobile	11
2.8 Étape 6 — Le build de développement, qu'on ne peut pas éviter	12
3 CHAQUE BIBLIOTHÈQUE, ET POURQUOI ELLE EST LÀ	13
3.1 Le paquet partagé	13
3.2 Le serveur	13
3.3 Le mobile	16
4 LES COMPOSANTS ET LA FAÇON DONT ILS SE TIENNENT	19
4.1 L'arborescence, commentée	19
4.2 Les fournisseurs, et pourquoi leur ordre compte	20
4.3 Le trajet complet d'une donnée	21
4.4 Le bus interne du serveur	21
5 LES ALGORITHMES DE CARTOGRAPHIE	22
5.1 La distance : la formule de haversine	22
5.2 La durée estimée, et pourquoi elle est marquée provisoire	23
5.3 Le lissage de la vitesse : la moyenne mobile exponentielle	24
5.4 Le rejet des sauts implausibles	24
5.5 Le filtre de position, en quatre passes	25
5.6 Le cadrage de la caméra	25
5.7 Les cercles de précision	26
5.8 Le décodage des itinéraires	26

5.9	Le style de carte-----	27
5.10	La recherche de proximité, côté base-----	28
5.11	La géométrie du service et la preuve d'arrivée-----	30
6	LE BACKEND, CHOIX PAR CHOIX-----	32
6.1	Le squelette d'Express-----	32
6.2	Les trois barrières de validation-----	32
6.3	L'authentification-----	33
6.4	Le canal temps réel, en détail-----	33
6.5	Le format des erreurs-----	34
7	LA BASE DE DONNÉES : LIRE ET MODIFIER LE SCHÉMA-----	35
7.1	Comment les migrations fonctionnent-----	35
7.2	Ajouter une migration-----	35
7.3	Le schéma, en vue d'ensemble-----	36
8	LES TESTS-----	37
8.1	Ce qui est couvert, et par quoi-----	37
8.2	Les tests géospatiaux, et pourquoi ils sont ignorés par défaut-----	37
8.3	Le test qui vérifie un plan d'exécution-----	37
8.4	Les scripts d'audit-----	38
9	LIVRER ET METTRE À JOUR-----	39
9.1	Les profils de construction-----	39
9.2	Les mises à jour à distance-----	39
10	LES PIÈGES, RASSEMBLÉS-----	40
10.1	Au démarrage-----	40
10.2	Sur la carte-----	40
10.3	Sur le réseau et le suivi-----	40
10.4	Sur la fidélité et les commissions-----	41
11	COMMANDES DE RÉFÉRENCE-----	42
11.1	Les constantes à connaître de mémoire-----	42
11.2	Où regarder en premier-----	43

TABLE DES FIGURES

Figure 1	Les quatre couches, leurs responsabilités et leurs dépendances externes-----	7
Figure 2	L'ordre d'installation, et les trois erreurs qui font perdre une soirée-----	8
Figure 3	Le pipeline complet, du capteur GPS au marqueur peint-----	22

Figure 4	Les deux modes de service, et la fonction unique qui les traduit en géométrie-----	30
Figure 5	Les quatorze tables, leurs clés, leurs contraintes et les index décisifs-----	36

VUE D'ENSEMBLE EN CINQ MINUTES

1.1 CE QUE LE SYSTÈME FAIT, MÉCANIQUEMENT

Un conducteur ouvre l'application. Son téléphone capte sa position, la filtre, et la garde. Il appuie sur SOS, décrit sa panne en trois écrans, et envoie. Le serveur enregistre la demande, interroge PostgreSQL pour trouver les garages dans un rayon, les classe, et renvoie tout dans la même réponse. Le conducteur choisit un garage ; le serveur pousse la demande au propriétaire de ce garage par un canal permanent. Celui qui doit se déplacer déclare son départ, et à partir de là son téléphone émet sa position toutes les quatre secondes tant qu'il bouge. Le serveur enregistre chaque point, calcule deux durées d'arrivée et les diffuse aux deux parties. Quand ils sont à moins de cent vingt mètres l'un de l'autre, chacun confirme. La base refuse de clore la demande tant que les deux confirmations n'y sont pas. Une fois close, un crédit de fidélité est écrit, et une ligne de commission est calculée à partir de la trace du voyageur.

Tout le reste — les écrans, les tables, les bibliothèques — sert cette séquence.

1.2 LES TROIS PAQUETS ET LA RÈGLE QUI LES GOUVERNE

Le dépôt est un monorepo npm workspaces. Trois paquets, et un seul sens de dépendance.

```
geocras/  
├─ packages/shared/      @geocras/shared      ne dépend de personne  
├─ apps/api/             @geocras/api         dépend de shared  
└─ apps/mobile/          @geocras/mobile      dépend de shared
```

La règle est qu'aucune flèche ne remonte. Le paquet partagé ignore l'existence du serveur et du mobile ; c'est ce qui lui permet d'être la source unique de vérité sans créer de cycle. Le serveur et le mobile ne se connaissent pas non plus : ils ne communiquent que par HTTP et par le canal temps réel, et ce qu'ils échangent est décrit dans le paquet partagé.

Retenez cette conséquence pratique : **`packages/shared` doit être compilé avant que quoi que ce soit d'autre puisse démarrer**. Les deux autres paquets importent son dossier `dist/`, pas ses sources. C'est la première cause d'échec au démarrage sur une machine neuve, et le message d'erreur — « Cannot find module @geocras/shared » — ne dit pas quoi faire.

1.3 LES QUATRE COUCHES

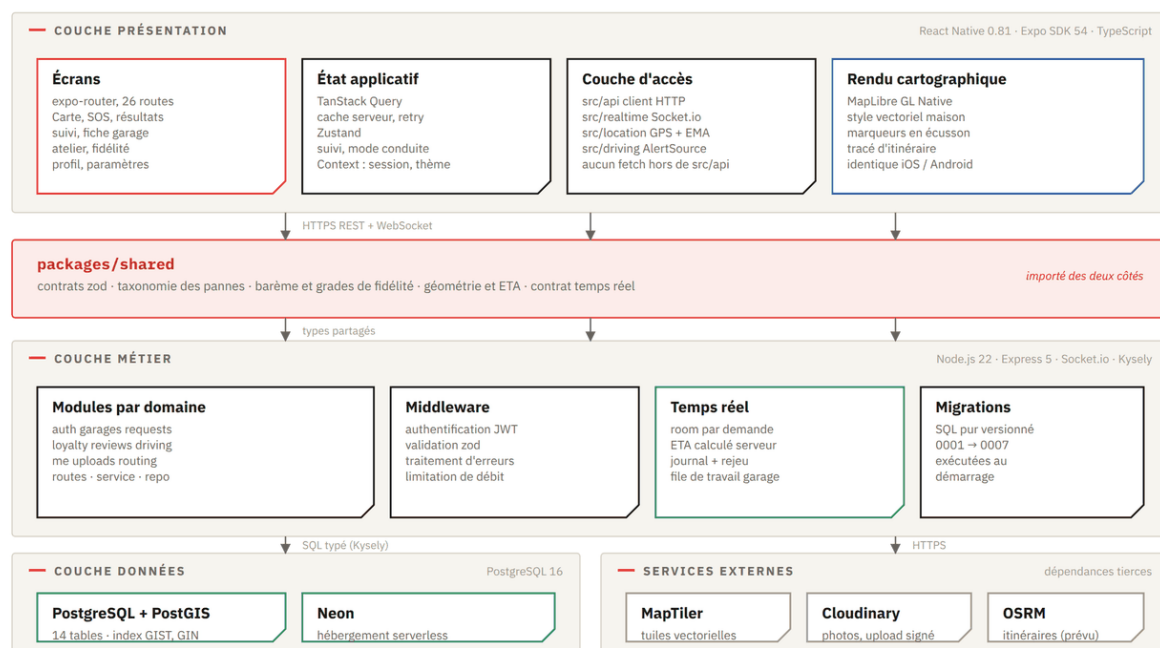


Figure 1 — Les quatre couches, leurs responsabilités et leurs dépendances externes

La couche de présentation est l'application mobile. Elle affiche, elle capte la position, elle garde l'état de l'écran. Elle ne décide rien qui touche à l'argent ou à la preuve.

Le paquet partagé n'est pas une couche au sens classique : il est traversé par les deux autres. Il porte les schémas de validation, la taxonomie des pannes, le barème de fidélité, la géométrie, le contrat du canal temps réel et le modèle de facturation.

La couche métier est le serveur. C'est lui qui valide, qui décide, qui calcule les durées d'arrivée, qui écrit le journal et qui juge la preuve d'arrivée.

La couche de données est PostgreSQL avec son extension géospatiale. Elle ne se contente pas de stocker : elle porte les règles métier qui ne doivent jamais céder, sous forme de contraintes.

INSTALLER LE PROJET, PAS À PAS

2.1 LA VUE D'ENSEMBLE DE L'INSTALLATION

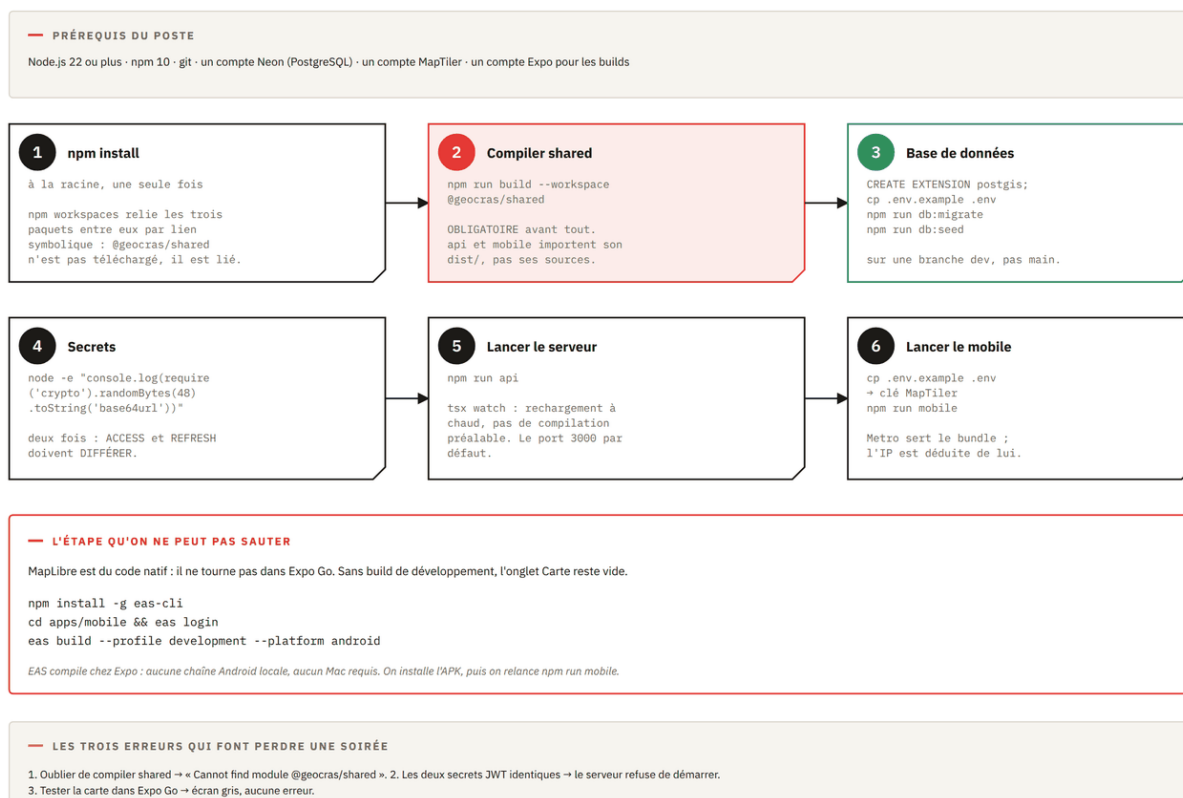


Figure 2 — L'ordre d'installation, et les trois erreurs qui font perdre une soirée

2.2 ÉTAPE 0 — CE QU'IL FAUT SUR LA MACHINE

Il faut Node.js en version 22 ou plus. Le champ engines du package.json racine l'exige, et ce n'est pas décoratif : le code utilise des fonctions qui n'existent pas avant. Il faut npm 10, qui vient avec Node 22, et git.

Il faut aussi trois comptes extérieurs, tous avec un palier gratuit suffisant pour développer. Un compte Neon pour la base PostgreSQL. Un compte MapTiler pour les tuiles de carte. Un compte Expo pour construire les versions installables.

VÉRIFIER AVANT D'ALLER PLUS LOIN

```

node --version      # doit afficher v22.x ou plus
npm --version       # doit afficher 10.x ou plus
git --version
  
```


2.3 ÉTAPE 1 — CLONER ET INSTALLER

1. Cloner le dépôt et se placer à la racine.
2. Lancer `npm install` **à la racine**, et nulle part ailleurs.

Une seule commande installe les trois paquets. C'est le propre des workspaces npm : la racine déclare `workspaces: ["apps/*", "packages/*"]`, npm résout les dépendances de tous les paquets ensemble, met le résultat dans un unique `node_modules` racine, et relie les paquets internes entre eux par lien symbolique. Quand `apps/api` importe `@geocras/shared`, il ne télécharge rien : il suit un lien vers `packages/shared`.

L'intérêt dépasse le confort. Une version de bibliothèque partagée par deux paquets n'existe qu'une fois, ce qui empêche la situation où le serveur et le mobile valident avec deux versions différentes de la même bibliothèque de schémas — un bogue qui se manifeste par des messages d'erreur incompréhensibles et qu'on met une journée à comprendre.

Ne jamais lancer `npm install` dans un sous-dossier

Un `npm install` dans `apps/api` crée un `node_modules` local qui masque celui de la racine, casse le lien symbolique vers le paquet partagé, et installe une copie séparée des dépendances communes. Si vous l'avez fait par erreur, supprimez le `node_modules` du sous-dossier et relancez l'installation depuis la racine.

2.4 ÉTAPE 2 — COMPILER LE PAQUET PARTAGÉ

```
npm run build --workspace @geocras/shared
```

C'est l'étape qu'on oublie. Elle produit `packages/shared/dist/`, que les deux autres paquets importent. Son `package.json` déclare `main: "./dist/index.js"` et `types: "./dist/index.d.ts"` : sans compilation, ces deux fichiers n'existent pas et tout import échoue.

Pendant une session de travail où vous modifiez le paquet partagé, lancez plutôt le mode veille dans un terminal à part :

```
npm run dev --workspace @geocras/shared
```

Il recompile à chaque sauvegarde. Sans lui, une modification du barème de fidélité reste invisible du serveur jusqu'à la prochaine compilation manuelle — et vous chercherez pourquoi votre changement « ne prend pas ».

2.5 ÉTAPE 3 — LA BASE DE DONNÉES

1. Créer un projet sur `neon.tech`, puis **une branche** ``dev``. Ne jamais développer contre `main`.
2. Ouvrir la console SQL de cette branche et activer l'extension géospatiale : `CREATE EXTENSION IF NOT EXISTS postgis;`
3. Copier `apps/api/.env.example` vers `apps/api/.env`.

4. Coller la chaîne de connexion de la branche dev dans DATABASE_URL.
5. Générer les deux secrets d'authentification, puis les coller dans JWT_ACCESS_SECRET et JWT_REFRESH_SECRET.
6. Appliquer le schéma : `npm run db:migrate`.
7. Charger les données de développement : `npm run db:seed`.

GÉNÉRER LES SECRETS

```
node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"
```

Lancez cette commande **deux fois** et utilisez deux valeurs différentes. Le serveur vérifie la longueur au démarrage — trente-deux caractères au minimum — mais il ne peut pas vérifier que vous n'avez pas collé la même valeur dans les deux champs. Or si les deux secrets sont identiques, un jeton de rafraîchissement devient un jeton d'accès valide, et la durée de vie courte de l'accès ne protège plus rien.

CE QUE FAIT LE SEED, ET CE QU'IL VOUS DIT

Le script insère trente garages à Yaoundé avec des positions, des services, des horaires et des notes variées. Puis il fait quelque chose d'inhabituel : il affiche le plan d'exécution de la requête de proximité.

Lisez cette sortie. Vous devez y trouver `garages_location_idx`. Si l'index géospatial n'apparaît pas, la requête centrale du produit parcourt toute la table — ce qui ne se voit pas avec trente garages, et s'effondre avec trois mille. C'est un contrôle de santé, pas un journal décoratif.

Un second jeu de données existe pour les essais de terrain :

```
npm run db:seed:ebolowa      # trois garages à Ebolowa
npm run db:seed:garagiste    # un compte propriétaire de garage
```

Ils sont dans des scripts séparés à dessein. Yaoundé est le parc de lancement ; Ebolowa est un jeu d'essai qu'on voudra retirer. Les mélanger rendrait la suppression impossible sans trier à la main.

2.6 ÉTAPE 4 — LANCER LE SERVEUR

```
npm run api
```

La commande passe par `tsx watch`, qui exécute TypeScript directement et recharge à chaque sauvegarde. Il n'y a pas de compilation préalable en développement.

Le serveur refuse de démarrer si la configuration est incomplète. C'est volontaire : le fichier `src/config/env.ts` valide toutes les variables d'environnement avec un schéma, et lève une erreur détaillée si l'une manque ou est douteuse. Une interface de programmation qui

démarre avec un secret par défaut est pire qu'une interface qui ne démarre pas — la panne est silencieuse et la faille est en production.

VÉRIFIER QUE LE SERVEUR RÉPOND

```
curl http://localhost:3000/health
curl "http://localhost:3000/garages/nearby?lat=3.848&lng=11.5021&sort=certified"
```

La première renvoie un état et une version. La seconde renvoie les garages classés autour du centre de Yaoundé — et elle est publique, donc elle marche sans jeton. Si elle renvoie une liste, la chaîne complète serveur-base-PostGIS fonctionne.

2.7 ÉTAPE 5 — LANCER LE MOBILE

1. Copier `apps/mobile/.env.example` vers `apps/mobile/.env`.
2. Créer une clé sur cloud.maptiler.com et la coller dans `EXPO_PUBLIC_MAPTILER_KEY`.
3. Laisser `EXPO_PUBLIC_API_URL` **vide** en développement.
4. Lancer `npm run mobile`.

Le troisième point demande une explication, parce qu'il est contre-intuitif. En développement, l'application tourne sur un téléphone : « localhost » y désigne le téléphone lui-même, pas votre machine. Le fichier `src/config/env.ts` récupère donc l'adresse du serveur Metro, qui est la bonne adresse dans la quasi-totalité des cas.

Le code utilise `||` et non `??` pour ce repli, et ce détail vaut d'être connu. Une chaîne vide n'est pas nullish : avec `??`, un `EXPO_PUBLIC_API_URL=` vide l'emporterait sur la déduction automatique, toutes les requêtes partiraient vers une adresse vide, et l'application afficherait « connexion impossible » sans qu'aucune variable ne paraisse mal réglée.

Tableau 1 — Les trois variables du mobile

Variable	Rôle	En développement
<code>EXPO_PUBLIC_MAPTILER_KEY</code>	Tuiles de carte	votre clé ; vide, la carte affiche « indisponible »
<code>EXPO_PUBLIC_API_URL</code>	Adresse du serveur	laisser vide, l'IP de Metro est déduite
<code>EXPO_PUBLIC_SUPPORT_PHONE</code>	Numéro du menu Assistance	valeur par défaut suffisante

Le préfixe `EXPO_PUBLIC_` veut dire ce qu'il dit

Ces variables sont inlinées dans le bundle : elles sont lisibles par quiconque décompresse l'application. On n'y met jamais de secret. La clé MapTiler y a sa place, parce qu'elle se restreint par identifiant d'application côté fournisseur ; un secret Cloudinary n'y a rien à faire, et c'est pour cela que la signature d'envoi des photographies est produite par le serveur.

2.8 ÉTAPE 6 — LE BUILD DE DÉVELOPPEMENT, QU'ON NE PEUT PAS ÉVITER

C'est le point de blocage principal du projet, et il faut le comprendre une fois pour ne plus s'y perdre.

MapLibre est du code natif. Expo Go, l'application qui permet de tester un projet Expo sans rien compiler, embarque un jeu fixe de modules natifs — et MapLibre n'en fait pas partie. Dans Expo Go, l'onglet Carte reste donc vide, sans message d'erreur. Ce n'est pas un bogue à chercher : c'est la conséquence normale de l'outil employé.

La solution est un **development build** : une version installable qui contient vos modules natifs, et dans laquelle le code JavaScript continue de se recharger à chaud comme dans Expo Go.

```
npm install -g eas-cli
cd apps/mobile
eas login
eas build --profile development --platform android
```

La compilation a lieu chez Expo, pas sur votre machine. Vous n'avez besoin ni d'Android Studio, ni d'un Mac, ni d'une chaîne de compilation locale — ce qui, pour un projet mené depuis Yaoundé, n'est pas un détail. À la fin, vous téléchargez un fichier installable, vous l'installez sur le téléphone, puis vous relancez `npm run mobile` : le build se connecte à votre serveur Metro comme le faisait Expo Go.

LE RISQUE À MESURER PENDANT CE PREMIER BUILD

Le fichier `app.json` active la nouvelle architecture de React Native. Le greffon Expo de MapLibre existe, mais sa compatibilité avec cette architecture n'est pas confirmée. Trois issues sont possibles et il faut les connaître d'avance.

- La carte s'affiche aux bonnes couleurs : rien à faire, le risque est éteint.
- Écran gris ou plantage au montage : repasser `newArchEnabled` à `false` dans `app.json`, reconstruire, réessayer. Le kit de développement en version 54 est la dernière à l'autoriser, c'est donc une dette avec une date de péremption.
- Toujours cassé : se replier sur `react-native-maps`, en acceptant un style limité sur iOS.

Profitez de ce premier build pour mesurer la fluidité : soixante images par seconde au déplacement et au zoom, avec vingt marqueurs, sur un Android de milieu de gamme. C'est le budget que le produit s'est fixé.

CHAQUE BIBLIOTHÈQUE, ET POURQUOI ELLE EST LÀ

Ce chapitre passe les dépendances une par une. Pour chacune : ce qu'elle fait, pourquoi elle a été choisie, et ce qui a été écarté. Une dépendance dont on ne sait pas dire ce qu'elle remplace est une dépendance qu'on ne saura pas retirer.

3.1 LE PAQUET PARTAGÉ

Il n'a qu'une seule dépendance de production, et c'est un signe de bonne santé.

Tableau 2 — Dépendances de packages/shared

Bibliothèque	Version	Rôle
zod	^3.25	Décrire et valider la forme des données, et en dériver les types TypeScript
typescript	~5.9	Compilateur, en dépendance de développement
vitest	^4.1	Exécuteur de tests, en dépendance de développement

POURQUOI ZOD, ET PAS UNE VALIDATION ÉCRITE À LA MAIN

Zod permet de déclarer une forme une seule fois et d'en tirer deux choses : un validateur exécutable et un type TypeScript. C'est ce doublet qui fait tout l'intérêt.

Le principe, en trois lignes

```
const schema = z.object({ lat: z.number(), lng: z.number() });
type Coordinates = z.infer<typeof schema>; // { lat: number; lng: number }
const result = schema.safeParse(donneesInconnues);
```

Sans cela, il faudrait écrire le type d'un côté et le contrôle de l'autre, et les deux divergeraient. Avec un type seul, TypeScript ne protège que la compilation : à l'exécution, une réponse réseau mal formée traverse tout sans obstacle. Zod comble exactement ce trou, à la frontière du système.

Le fait qu'il vive dans le paquet partagé est le vrai gain. La même définition sert au mobile pour désactiver un bouton et au serveur pour refuser une requête. La règle est écrite une fois.

3.2 LE SERVEUR

Tableau 3 — Dépendances de production de apps/api

Bibliothèque	Version	Rôle	Ce qui a été écarté
--------------	---------	------	---------------------

express	^5.1	Serveur HTTP, routage, intergiciels	Fastify, plus rapide mais moins documenté ; Nest, trop de structure imposée
kysely	^0.27	Constructeur de requêtes SQL typé	Prisma, qui ne sait pas typer les colonnes géographiques
pg	^8.13	Pilote PostgreSQL, pool de connexions	rien : c'est le pilote de référence
socket.io	^4.8	Canal bidirectionnel avec reconnexion et repli de transport	WebSocket brut, qui obligerait à réécrire la reconnexion
zod	^3.25	Validation des entrées et de l'environnement	Joi, sans inférence de types
jsonwebtoken	^9.0	Signature et vérification des jetons d'accès	une session en base, qui coûterait une lecture par requête
bcryptjs	^3.0	Hachage des mots de passe	bcrypt natif, qui complique la compilation ; argon2, préférable mais moins portable
helmet	^8.0	En-têtes HTTP de sécurité	les écrire à la main, et en oublier un
cors	^2.8	Contrôle des origines navigateur	rien
express-rate-limit	^7.5	Limitation de débit sur l'inscription et la connexion	rien
pino et pino-http	^9.6 / ^10.4	Journalisation structurée, très rapide	Winston, plus lent et plus verbeux
dotenv	^16.4	Charger le fichier .env en développement	rien

Tableau 4 — Dépendances de développement de apps/api

Bibliothèque	Rôle
tsx	Exécute TypeScript directement, avec rechargement à chaud. C'est lui derrière <code>npm run api</code>
vitest	Exécuteur de tests
supertest	Envoie de vraies requêtes HTTP à l'application Express, sans ouvrir de port
pino-pretty	Rend les journaux lisibles en développement
@types/*	Déclarations de types pour les bibliothèques qui

	n'en fournissent pas
--	----------------------

EXPRESS 5 PLUTÔT QUE 4

La version 5 apporte une chose qui change la vie du code : les erreurs levées dans un gestionnaire asynchrone sont automatiquement transmises à l'intergiciel d'erreur. En version 4, un `throw` dans une fonction `async` était silencieusement perdu, et il fallait envelopper chaque route dans un utilitaire de capture. Tout le code du projet en dépend — les routes appellent des services asynchrones et laissent remonter les erreurs.

KYSELY, ET LE TYPE VOLONTAIREMENT PÉNIBLE

Kysely construit du SQL en TypeScript, avec un résultat typé. Contrairement à un ORM complet, il ne cache pas le SQL : on lit ce qui part vers la base.

Le point décisif est le géospatial. Un ORM classique ne sait pas typer une colonne `GEOGRAPHY`, et le contournement habituel consiste à la déclarer non supportée puis à passer par des requêtes brutes non typées pour toute la moitié géographique du produit — c'est-à-dire pour son cœur.

Le projet va plus loin : le type qui représente un point est fait pour être désagréable.

src/db/types.ts, l'idée

```
// En écriture : n'accepte qu'un fragment SQL, donc on passe forcément
// par pointFromLatLng().
// En lecture : rend une chaîne binaire inexploitable, donc on est forcé
// de projeter ST_Y et ST_X explicitement.
```

C'est pénible à l'usage, et c'est le but. On ne peut pas se tromper par distraction, et l'erreur classique — écrire une latitude là où on attend une longitude — devient impossible à commettre en silence.

SOCKET.IO PLUTÔT QUE WEBSOCKET BRUT

Trois raisons, dans l'ordre d'importance pour ce produit précis.

D'abord le repli de transport. Quand la connexion permanente ne s'établit pas — réseau filtrant, proxy, 2G capricieuse — Socket.io bascule tout seul sur une technique plus tolérante. C'est exactement ce qui sauve une session de suivi au Cameroun, et c'est ce qu'il faudrait réécrire à la main avec un WebSocket brut.

Ensuite la reconnexion, avec temporisation croissante, gérée sans code de notre part.

Enfin les salles, qui donnent gratuitement le découpage dont le produit a besoin : une salle par demande d'assistance, et une salle par compte utilisateur.

3.3 LE MOBILE

Tableau 5 — Le socle

Bibliothèque	Version	Rôle
expo	~54.0	Le cadre : modules natifs préconfigurés, chargement de polices, construction distante
react-native	0.81.5	Le moteur de rendu natif
react	19.1	La bibliothèque de composants
expo-router	~6.0	Routage par arborescence de fichiers, typé
typescript	~5.9	Typage

LE ROUTAGE PAR FICHIERS, EN UNE PHRASE

Avec expo-router, l'arborescence du dossier app/ **est** la navigation. Un fichier devient un écran, un dossier devient un groupe, et les parenthèses créent un groupe qui n'apparaît pas dans l'URL.

Ce que la structure signifie

app/index.tsx	→ l'écran d'ouverture (splash)
app/(drawer)/_layout.tsx	→ le tiroir latéral englobe tout ce qui suit
app/(drawer)/(tabs)/_layout.tsx	→ la barre d'onglets, à l'intérieur du tiroir
app/(drawer)/(tabs)/carte.tsx	→ l'onglet Carte
app/garage/[id].tsx	→ fiche d'un garage, id dynamique
app/suivi/[requestId].tsx	→ écran de suivi

Les fichiers `_layout.tsx` sont la clé : ils décrivent ce qui **entoure** les écrans d'un niveau. C'est là que vivent les fournisseurs de contexte, et l'ordre dans lequel ils s'imbriquent a des conséquences réelles, détaillées au chapitre suivant.

Tableau 6 — La carte et la géolocalisation

Bibliothèque	Rôle
@maplibre/maplibre-react-native	Rendu vectoriel de la carte. Code natif, d'où l'obligation du build de développement
expo-location	Accès au récepteur de positionnement, permissions, suivi continu

Tableau 7 — L'état applicatif

Bibliothèque	Rôle	Ce qu'elle ne fait pas
@tanstack/react-query	Cache des réponses serveur, réessai, fraîcheur, persistance hors ligne	jamais de position temps réel : on invaliderait le cache en boucle

zustand	État qui change plusieurs fois par seconde : suivi, mode conduite	pas de cache serveur
@react-native-async-storage/async-storage	Stockage local des préférences et des brouillons	pas de secret
expo-secure-store	Stockage chiffré des jetons d'authentification	pas de gros volume

Tableau 8 — L'interface

Bibliothèque	Rôle
react-native-safe-area-context	Marges des encoches et barres système
react-native-gesture-handler	Gestes natifs : feuilles glissantes, tiroir
react-native-reanimated	Animations exécutées sur le fil natif, donc fluides même quand JavaScript est occupé
react-native-screens	Écrans natifs sous la navigation, pour la mémoire et les transitions
react-native-svg	Formes vectorielles : l'angle coupé, les écussons, les étoiles
expo-linear-gradient	Dégradés, notamment les voiles au-dessus de la carte
@react-navigation/drawer	Le tiroir latéral
expo-haptics	Retour tactile sur les actions importantes

Tableau 9 — Les polices et les services de l'appareil

Bibliothèque	Rôle
@expo-google-fonts/bebas-neue	Titres, boutons, libellés de section
@expo-google-fonts/ibm-plex-sans et -condensed	Texte courant
@expo-google-fonts/ibm-plex-mono	Toute donnée mesurée
@expo-google-fonts/inter	Échelle historique, en cours de remplacement
expo-font	Chargement des polices avant le premier rendu
expo-splash-screen	Masque l'écran natif jusqu'à ce que tout soit prêt
expo-image-picker	Photographie de la panne
expo-notifications	Notifications poussées, routes prêtes, branchement à faire
expo-keep-awake	Empêche l'extinction de l'écran en mode conduite
expo-localization	Détecte la langue du téléphone
expo-linking	Liens profonds, appels téléphoniques, ouverture d'applications externes
expo-constants	Accès à la configuration, dont l'adresse du serveur Metro

<code>expo-status-bar</code>	Couleur de la barre d'état selon le thème
<code>expo-dev-client</code>	Ce qui rend le build de développement possible
<code>socket.io-client</code>	Le pendant client du canal temps réel

Pourquoi deux familles de polices chargées en même temps

La refonte typographique remplace Inter par IBM Plex écran par écran. Pendant la bascule, les deux familles sont chargées : un écran non repris demande encore une variante Inter, et retirer la famille avant la fin de la passe le ferait tomber en police système sans qu'aucune erreur ne le signale. Inter partira quand le dernier écran sera repris.

LES COMPOSANTS ET LA FAÇON DONT ILS SE TIENNENT

4.1 L'ARBORESCENCE, COMMENTÉE

packages/shared/src — ce qui ne doit jamais diverger

contracts/	
auth.ts	inscription, connexion, jetons
common.ts	coordonnées, pagination, identifiants
garages.ts	fiche et résumé de garage, tris, services
requests.ts	demande d'assistance, statuts, libellés d'état
jobs.ts	file de travail du garagiste, action suivante
realtime.ts	noms d'événements, cadences, seuils de proximité
reviews.ts	avis
routing.ts	itinéraire
driving.ts	session de conduite
loyalty.ts	contrats de fidélité
uploads.ts	signature d'envoi
taxonomy.ts	véhicules, pannes ordonnées, modes de service
loyalty.ts	barème, grades, garde-fous anti-fraude
billing.ts	géométrie du service, preuve d'arrivée, tarifs
geo.ts	haversine, durée estimée, lissage

apps/api/src — découpé par domaine, pas par couche

app.ts	assemblage d'Express, intergiciels, montage des routes
index.ts	démarrage, serveur HTTP, attachement du canal temps réel
config/env.ts	validation de l'environnement au démarrage
db/	
client.ts	pool PostgreSQL et instance Kysely
types.ts	types des tables, dont le type géographique hostile
geo.ts	pointFromLatLng, seul endroit qui écrit du ST_SetSRID
migrate.ts	exécuteur de migrations
middleware/	
auth.ts	requireAuth, lecture du jeton
validate.ts	validateBody, validateQuery
error.ts	format d'erreur unique, 404
modules/	
auth/	routes, service, jetons
garages/	routes, service, repo, requête de proximité
requests/	routes, service, repo, suivi, preuve, commission
reviews/	service
loyalty/	service
driving/	routes, service
me/	routes, service (profil, garage, véhicules, fidélité)
uploads/	signature Cloudinary
routing/	appel OSRM, décodage polyline
realtime/	
io.ts	salles, authentification du socket, rejeu
bus.ts	émetteur interne : les services publient, io écoute
seed/	yaounde, ebolowa, garage-owner

Le découpage par domaine mérite d'être défendu, parce qu'il n'est pas le plus répandu. Ajouter un champ à la demande d'assistance touche `modules/requests/` et rien d'autre : la route, le service et l'accès aux données sont côte à côte. Un découpage horizontal — tous les contrôleurs ensemble, tous les services ensemble — aurait imposé d'ouvrir quatre dossiers pour un seul champ, et de garder en tête la correspondance entre eux.

apps/mobile/src — par responsabilité

<code>api/</code>	client HTTP, hooks de requête, jetons, joignabilité
<code>auth/</code>	AuthProvider : session, connexion, déconnexion
<code>location/</code>	LocationProvider, filtre de position, origine stable
<code>map/</code>	MapCanvas, style, marqueurs, tracé, cercles, cadrage
<code>realtime/</code>	socket, useTracking, useJobFeed
<code>tracking/</code>	écrans de suivi : LiveTracking, DriveToGarage, panneaux
<code>sos/</code>	déclaration de panne : sélecteurs, photo, résultats
<code>jobs/</code>	côté garagiste : file, fiche, urgence, horloge d'attente
<code>garage/</code>	fiche garage : avis, horaires, photos, aperçu d'itinéraire
<code>driving/</code>	mode conduite : source d'alertes, moteur, compteurs
<code>loyalty/</code>	fidélité : paliers, squelettes de chargement
<code>history/</code>	historique : cartes de demande, « redemander »
<code>theme/</code>	jetons, typographie, fournisseur de thème
<code>ui/</code>	primitives : ChamferView, Text, Button, SectionLabel, Stars
<code>i18n/</code>	fournisseur et traductions
<code>settings/</code>	préférences
<code>updates/</code>	mise à jour à distance

4.2 LES FOURNISSEURS, ET POURQUOI LEUR ORDRE COMPTE

Le fichier `app/_layout.tsx` empile les fournisseurs de contexte. Cet ordre n'est pas arbitraire : chaque niveau peut consommer ce que les niveaux au-dessus fournissent, jamais l'inverse.

L'imbrication, de l'extérieur vers l'intérieur

SafeAreaProvider	les marges système, dont tout dépend
QueryClientProvider	le cache réseau
ThemeProvider	thème clair/sombre, jetons de couleur
I18nProvider	langue et libellés
AuthProvider	session et jetons — a besoin du cache réseau
LocationProvider	position — a besoin de la session pour émettre
Stack	les écrans

Deux conséquences pratiques. Le cache réseau est au-dessus de la session parce que la connexion est elle-même une requête réseau : l'inverse créerait une dépendance circulaire. Et la position est en dessous de la session parce qu'émettre une position exige d'être authentifié — le fournisseur de position interroge la session pour savoir s'il doit émettre.

Si vous ajoutez un fournisseur, posez-vous une seule question : de quoi a-t-il besoin ? Il se place en dessous de ce qu'il consomme.

4.3 LE TRAJET COMPLET D'UNE DONNÉE

Suivons un seul geste — l'appui sur SOS — à travers toutes les couches. C'est l'exercice qui fait le plus pour la compréhension d'un système.

1. **L'écran** `app/sos/declarer.tsx` collecte les réponses. Il ne fait aucun appel réseau lui-même.
2. **Le contrat** `packages/shared/src/contracts/requests.ts` valide la forme localement. Le bouton « Continuer » reste inerte tant que le schéma refuse. La règle qui exige un libellé libre pour un véhicule de catégorie ouverte vit ici, et nulle part ailleurs.
3. **Le hook** `src/api/hooks.ts` expose `useCreateRequest()`. C'est une mutation de la bibliothèque de cache : elle gère l'état d'envoi, le réessai et l'invalidation de ce qui devient périmé.
4. **Le client** `src/api/client.ts` ajoute le jeton d'accès, sérialise, et lance la requête. Aucun composant n'appelle `fetch` directement — c'est une règle du projet, et c'est ce qui a rendu le mode dégradé implémentable.
5. **La route** `apps/api/src/modules/requests/requests.routes.ts` reçoit. L'intergiciel `validateBody` applique **le même schéma** qu'à l'étape 2. Un client modifié ne passe pas.
6. **Le service** `requests.service.ts` décide : il vérifie qu'aucune demande n'est active, écrit la demande, inscrit l'événement fondateur au journal avec le mode de service, puis appelle la recherche de garages.
7. **L'accès aux données** `garages.repo.ts` construit la requête de proximité en SQL typé, avec le filtre géospatial avant le tri.
8. **La base** applique ses contraintes. Si le code se trompe — un véhicule immobilisé avec le mode « je vais au garage » — la ligne est refusée ici, dernier rempart.
9. **La réponse** remonte : la demande et les garages classés, dans un seul aller-retour.
10. **Le canal temps réel** `realtime/io.ts` pousse la file de travail au propriétaire du garage, si un garage a été retenu.
11. **Le cache** côté mobile enregistre la réponse. L'écran de résultats la lit sans nouvel appel.

Onze étapes, et une seule règle métier écrite une seule fois — appliquée deux fois, à l'étape 2 et à l'étape 5, parce que les deux extrémités importent le même fichier.

4.4 LE BUS INTERNE DU SERVEUR

Un détail d'architecture qui évite un enchevêtrement : les services métier ne connaissent pas le canal temps réel.

Quand `requests.service.ts` fait avancer une demande, il ne parle pas à `Socket.io`. Il publie sur un émetteur interne, `realtime/bus.ts`. C'est `realtime/io.ts` qui écoute ce bus et traduit en émissions vers les salles concernées.

L'intérêt est net. Les services restent testables sans démarrer de serveur de socket. Et le jour où un second canal de diffusion apparaît — notifications poussées, journal d'audit externe — il s'abonne au même bus sans qu'aucun service ne change.

LES ALGORITHMES DE CARTOGRAPHIE

Ce chapitre est le cœur technique du projet. Il détaille chaque calcul, dans l'ordre où la donnée les traverse.

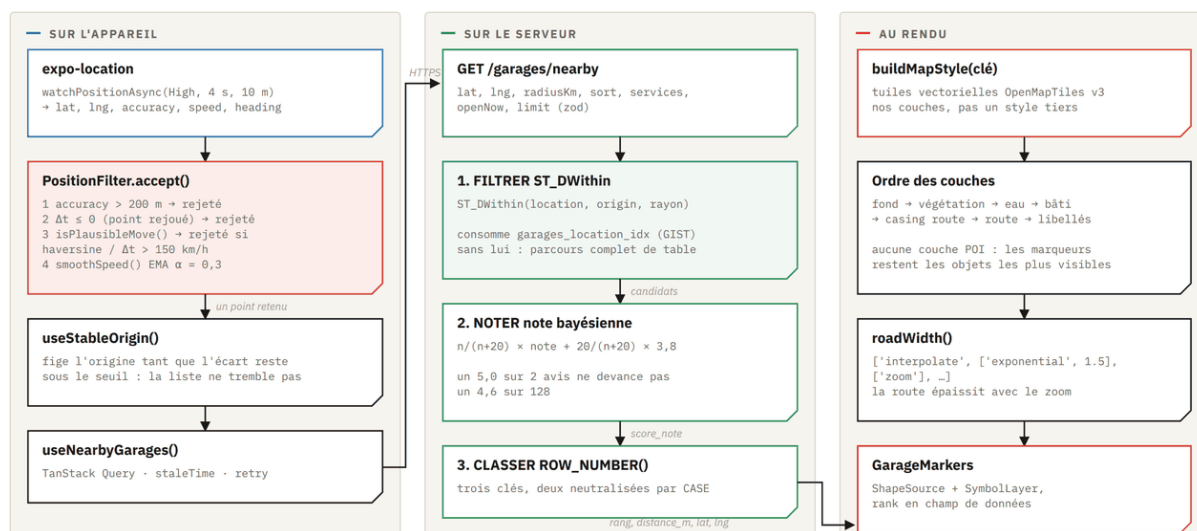


Figure 3 — Le pipeline complet, du capteur GPS au marqueur peint

5.1 LA DISTANCE : LA FORMULE DE HAVERSINE

Deux points sur une sphère, une distance en mètres. C'est la brique sur laquelle tout le reste repose.

packages/shared/src/geo.ts

```
const EARTH_RADIUS_M = 6_371_008.8;

h = sin²(Δφ/2) + sin²(Δλ/2) · cos φ₁ · cos φ₂
d = 2 R · asin( min(1, vh) )
```

Trois choses sont à comprendre.

Le rayon employé, 6 371 008,8 mètres, est le rayon moyen de la Terre au sens de l'Union astronomique internationale. À l'échelle d'une ville, choisir un autre rayon plausible change le résultat de quelques mètres — c'est-à-dire moins que l'imprécision du récepteur.

La formule est celle de haversine et non la loi sphérique des cosinus, qui est plus simple mais perd sa précision sur les courtes distances à cause d'arrondis en virgule flottante. Or les courtes distances sont exactement le cas d'usage : on mesure des centaines de mètres, pas des continents.

Le `min(1, ...)` avant l'arcsinus n'est pas une coquetterie. Pour deux points identiques ou presque, l'accumulation d'erreurs d'arrondi peut produire une valeur très légèrement supérieure à un, et `asin` d'un nombre supérieur à un rend `NaN`. Ce `NaN` se propagerait ensuite dans la durée estimée, dans le cadrage de la caméra, et dans la preuve d'arrivée — sans qu'aucune erreur ne soit levée. Une ligne de garde contre un mode de panne silencieux.

Haversine côté client, PostGIS côté serveur

Le serveur n'utilise pas cette fonction pour la recherche de proximité : il emploie `ST_DWithin` et `ST_Distance` sur des colonnes de type `GEOGRAPHY`, qui tiennent compte de l'ellipsoïde et surtout consomment l'index géospatial. Haversine sert là où il n'y a pas de base à interroger : sur le téléphone, et dans le calcul de la preuve d'arrivée à partir d'une trace déjà chargée en mémoire.

5.2 LA DURÉE ESTIMÉE, ET POURQUOI ELLE EST MARQUÉE PROVISOIRE

packages/shared/src/geo.ts

```
ETA_CALIBRATION = {
  detourFactor: 1.35,    distance routière / distance à vol d'oiseau
  urbanSpeedKmh: 22,     vitesse en tissu urbain dense
  walkingSpeedKmh: 4.5,
}

estimateEtaMinutes(d, mode) = max(1, round( d × 1,35 / 1000 / v × 60 ))
```

Le facteur de détour est l'aveu d'une approximation. Une distance à vol d'oiseau n'est pas une distance parcourue : il faut suivre des rues, contourner des îlots, respecter des sens interdits. Le rapport observé entre les deux, dans un tissu urbain dense et peu maillé, tourne autour de 1,35. Cette valeur est une calibration pour Yaoundé, à réviser sur les données réelles de la table des relevés de position.

La vitesse de vingt-deux kilomètres par heure peut sembler basse. Elle inclut les arrêts, les carrefours et la circulation — c'est une vitesse moyenne de trajet, pas une vitesse de roulage.

Le `max(1, ...)` évite d'annoncer « arrive dans 0 minute », qui se lit comme un défaut d'affichage plutôt que comme une bonne nouvelle.

Cette fonction est explicitement transitoire. Le fichier porte un avertissement en tête, et l'interface marque la valeur comme provisoire. La cible est un serveur de routage OSRM auto-hébergé, désigné par la variable `OSRM_URL` — et rien d'autre ne bougera le jour de la bascule.

Le défaut d'`OSRM_URL` est un serveur public de démonstration

Il dépanne en développement, mais sa politique d'usage interdit la production : aucune garantie de disponibilité, quota partagé, et chaque requête envoie à un tiers la position d'un garagiste et le lieu d'une panne. C'est une variable à changer avant toute mise en service réelle.

5.3 LE LISSAGE DE LA VITESSE : LA MOYENNE MOBILE EXPONENTIELLE

packages/shared/src/geo.ts

```
SPEED_SMOOTHING_ALPHA = 0.3

smoothSpeed(précédent, échantillon) =
  précédent === null ? échantillon
    : 0,3 × échantillon + 0,7 × précédent
```

Le principe est simple : chaque nouvelle mesure ne compte que pour trois dixièmes, les sept dixièmes restants venant de l'historique. La valeur affichée bouge donc lentement, même quand la mesure saute.

Le choix de trois dixièmes est un arbitrage. Plus haut, la valeur suit mieux les accélérations réelles mais tremble ; plus bas, elle devient très stable mais met plusieurs relevés à réagir à un vrai changement d'allure. À une émission toutes les quatre secondes, 0,3 donne une réponse en une quinzaine de secondes — assez vif pour un usage routier, assez calme pour que le chiffre ne clignote pas.

Pourquoi lisser ? Parce que la durée d'arrivée se calcule à partir de la vitesse, et qu'une vitesse brute fait sauter le chiffre d'un relevé à l'autre. Un temps d'arrivée qui passe de huit à quinze puis à six minutes est plus anxiogène qu'aucun temps du tout : l'utilisateur cesse de le croire, et avec lui il cesse de croire le reste de l'écran.

5.4 LE REJET DES SAUTS IMPLAUSIBLES

packages/shared/src/geo.ts

```
MAX_PLAUSIBLE_SPEED_MPS = 150 / 3,6      ≈ 41,7 m/s

isPlausibleMove(de, vers, Δt) =
  Δt > 0 ET haversine(de, vers) / Δt ≤ 41,7
```

Le raisonnement est physique. En ville camerounaise, un déplacement impliquant plus de cent cinquante kilomètres par heure entre deux relevés n'est pas une voiture : c'est une dérive du récepteur, typiquement causée par un signal réfléchi entre deux immeubles ou par une position calculée sur trop peu de satellites.

Le seuil est volontairement généreux. Un seuil serré — quatre-vingts kilomètres par heure, par exemple — rejetterait des déplacements réels sur les axes rapides, et l'utilisateur verrait son point rester collé. Mieux vaut laisser passer une aberration modérée que censurer un mouvement authentique.

Le test $\Delta t > 0$ traite un cas réel : un horodatage identique ou antérieur au précédent signale un relevé rejoué par le système d'exploitation. Sans cette garde, on diviserait par zéro.

5.5 LE FILTRE DE POSITION, EN QUATRE PASSES

La classe `PositionFilter` dans `apps/mobile/src/location/filter.ts` assemble les briques précédentes en une chaîne de décision. Chaque relevé du récepteur y passe.

1. **Précision.** Au-delà de deux cents mètres, la mesure est écartée. Le seuil est généreux à dessein : masquer un point à ± 150 m laisserait l'utilisateur sans position du tout, alors que ± 150 m reste une information exploitable pour chercher un garage.
2. **Horodatage.** Un écart nul ou négatif signale un relevé rejoué. On garde le précédent.
3. **Plausibilité.** Le déplacement est confronté au seuil de cent cinquante kilomètres par heure. Rejeté s'il le dépasse.
4. **Lissage.** La vitesse retenue est la moyenne mobile exponentielle de la précédente et de l'échantillon.

Une décision explicite mérite d'être soulignée : **on ne filtre pas sur la précision seule**. Un point à quarante mètres près reste la meilleure information disponible quand c'est tout ce qu'on a. Le produit affiche d'ailleurs cette précision réelle plutôt qu'une valeur rassurante — un badge « ± 5 m » qui mentirait détruirait la confiance dans tout le reste de l'écran.

5.6 LE CADRAGE DE LA CAMÉRA

Après avoir des points, il faut décider ce que la caméra montre. Deux fonctions, dans `apps/mobile/src/map/bounds.ts`.

```
boundsOf(points) → [ouest, sud, est, nord] | null
MIN_SPAN_DEGREES = 0,0004 ≈ 40 mètres
isDegenerate(b) → vrai si les DEUX côtés sont sous le seuil
```

L'ordre des quatre nombres suit celui qu'attend MapLibre, et c'est délibéré : aucun appelant n'a à réarranger quatre valeurs. C'est exactement le genre d'inversion latitude/longitude qui envoie une caméra au large du golfe de Guinée sans qu'aucune erreur ne soit levée.

Les points non finis sont **écartés** plutôt que propagés. Un seul NaN dans une géométrie suffirait à rendre les quatre bornes indéfinies, et la fonction de cadrage échouerait alors en silence : la caméra ne bougerait pas, et rien à l'écran ne dirait pourquoi.

Le test de dégénérescence traite un cas qui arrive à chaque intervention. Quand le dépanneur atteint la panne, son point de départ et sa destination se confondent. Cadrer un rectangle de hauteur nulle envoie la caméra en butée de zoom, et l'écran n'affiche plus qu'un carré de bitume. La condition exige que les **deux** côtés soient sous le seuil : un trajet parfaitement nord-sud a une largeur nulle et reste parfaitement cadrable.

5.7 LES CERCLES DE PRÉCISION

apps/mobile/src/map/circle.ts

```
metersPerDegreeLat = 111 320
metersPerDegreeLng = 111 320 × cos(latitude)

pour i de 0 à 48 :
  angle = i/48 × 2π
  point = [ lng + r·cos(angle)/metersPerDegreeLng,
            lat + r·sin(angle)/metersPerDegreeLat ]
```

Deux points de conception importent ici.

C'est un **polygone géographique**, pas un cercle d'écran. Le rayon représente une distance sur le terrain — la précision du récepteur, la zone d'incertitude d'une position masquée — et doit donc grandir quand on dézoome. Un cercle à rayon fixe en pixels afficherait une précision constante à tous les niveaux de zoom, c'est-à-dire exactement le contraire de ce que ces cercles servent à dire.

La projection est plate, et c'est assumé. Le cosinus de la latitude corrige le rétrécissement des degrés de longitude quand on s'éloigne de l'équateur. À l'échelle du kilomètre et sous l'équateur, l'écart avec une géodésique exacte est de l'ordre du centimètre. Ce qu'on dessine est une incertitude, pas un cadastre.

Quarante-huit segments donnent un cercle visuellement lisse sans alourdir la géométrie. En dessous d'une vingtaine, le polygone se voit.

5.8 LE DÉCODAGE DES ITINÉRAIRES

OSRM peut rendre sa géométrie en GeoJSON, mais le projet demande du **polyline6**. La raison est un facteur dix : un trajet urbain d'un kilomètre pèse environ trois cents octets encodé, contre deux à trois kilooctets en GeoJSON, pour un tracé identique. Sur un forfait de données compté, la différence n'est pas théorique.

L'algorithme est celui de Google, connu sous le nom d'*encoded polyline algorithm format*. Il repose sur trois idées empilées.

1. **Différences successives.** On n'encode pas les coordonnées absolues mais l'écart avec le point précédent. Des écarts sont de petits nombres, donc courts à écrire.
2. **Zig-zag.** Un entier signé est transformé en entier non signé en repliant les négatifs sur les impairs : le bit de poids faible porte le signe. Cela évite de gaspiller les bits de poids fort pour représenter un signe.
3. **Groupe de cinq bits.** Le nombre est découpé en tranches de cinq bits, chacune écrite comme un caractère imprimable. Le sixième bit signale qu'une tranche suit.

Le suffixe « 6 » désigne le facteur d'échelle : un million au lieu de cent mille, soit une précision de dix centimètres au lieu d'un mètre.

Un piège mérite le détour, parce qu'il est classique et silencieux. **OSRM encode la latitude d'abord**, alors que GeoJSON et MapLibre attendent la longitude d'abord. L'inversion est faite dans `decodePolyline6`, au seul endroit du code qui connaît la convention d'OSRM. La laisser fuir plus loin produirait une polygline au milieu de l'océan Atlantique — sans aucune erreur levée, puisque les deux nombres sont valides.

Notez aussi que le décodage a lieu **sur le serveur**. Le mobile reçoit des coordonnées prêtes à peindre. C'est notre serveur qui absorbe le format d'OSRM, ce qui permettra de changer de moteur de routage sans toucher à l'application.

5.9 LE STYLE DE CARTE

Le projet ne charge pas un style tout fait. Il consomme les **tuiles vectorielles** de MapTiler et déclare ses propres couches, dans `apps/mobile/src/map/style.ts`.

C'est la seule façon d'obtenir exactement le fond `#EFEFE2` et les routes `#FEFBF0` sur casing `#E2DDD1` des maquettes : un style tiers impose ses couleurs, et l'identité visuelle est déclarée non négociable.

L'ordre des couches, du fond vers l'avant

<code>background</code>	l'aplat crème
<code>landcover-vegetation</code>	végétation
<code>park</code>	parcs
<code>water, waterway</code>	eau et cours d'eau
<code>building</code>	bâti
<code>road-*-casing</code>	le contour sombre des routes (mineures, secondaires, majeures)
<code>road-*</code>	le remplissage clair, par-dessus le casing
<code>road-label</code>	noms de rues
<code>place-label</code>	quartiers
<code>locality-label</code>	localités

La technique du **casing** est ce qui donne aux routes leur aspect dessiné : on peint d'abord un trait large et sombre, puis un trait plus fin et clair par-dessus. La différence de largeur crée un liseré qui sépare la route du fond. C'est pourquoi chaque niveau de route apparaît deux fois dans la liste, et pourquoi tous les casings viennent avant tous les remplissages — sinon un casing de route majeure recouvrirait le remplissage d'une route mineure à son croisement.

La largeur de route, interpolée par zoom

```
['interpolate', ['exponential', 1.5], ['zoom'], z1, w1, z2, w2, ...]
```

L'interpolation exponentielle de base 1,5 fait grossir la route plus vite aux zooms élevés qu'une interpolation linéaire. C'est ce qui correspond à la perception : en s'approchant, on s'attend à ce que la rue occupe proportionnellement plus de place.

Deux absences sont volontaires. **Aucune couche de points d'intérêt** : le cahier des charges interdit les repères parasites, et les seuls objets qui doivent capter l'œil sont les marqueurs de garages. Et aucune couleur vive en dehors du rouge de l'identité.

Un écart connu sur les polices de carte

Les glyphes vectoriels d'Inter ne sont pas disponibles chez MapTiler. Les étiquettes de rues utilisent donc Noto Sans, alors que l'interface est en IBM Plex. Pour aligner les deux, il faudra téléverser la police dans MapTiler Cloud et changer l'URL `glyphs`. Sans conséquence sur la règle « chiffres en mono », qui porte sur l'interface et non sur le rendu de la carte.

5.10 LA RECHERCHE DE PROXIMITÉ, CÔTÉ BASE

C'est la requête centrale du produit, et elle repose sur trois mécanismes de PostgreSQL.

LE TYPE GÉOGRAPHIQUE

Les positions sont stockées en `GEOGRAPHY(POINT, 4326)` et non en `GEOMETRY`. La différence est décisive : le type géographique rend directement des mètres en tenant compte de la courbure terrestre, sans conversion manuelle ni choix de projection à justifier. Sur un produit dont la fonction centrale est une distance en mètres, c'est le type qui dit la vérité par défaut.

Le nombre 4326 désigne le système de référence WGS 84, celui des récepteurs GPS.

L'INDEX GIST, ET POURQUOI LE FILTRE DOIT PRÉCÉDER LE TRI

```
CREATE INDEX garages_location_idx ON garages USING GIST (location);
CREATE INDEX garages_certified_location_idx ON garages USING GIST (location)
  WHERE is_active AND certified;
CREATE INDEX garages_services_idx ON garages USING GIN (services);
```

Un index GIST — *Generalized Search Tree* — organise l'espace en rectangles englobants emboîtés. Interroger « quels points sont à moins de dix kilomètres d'ici » revient à descendre l'arbre en éliminant des branches entières, au lieu de mesurer la distance de chaque ligne.

Encore faut-il que la requête le lui permette, et c'est là l'invariant à ne jamais casser :

Ce qui consomme l'index

```
WHERE ST_DWithin(g.location, $origine, $rayon)    ← filtre, indexable
ORDER BY distance_m                               ← tri, sur le résultat filtré
```

Ce qui ne le consomme pas

```
ORDER BY ST_Distance(g.location, $origine)       ← calcule pour TOUTE la table
```

La seconde forme est correcte : elle rend les bons garages dans le bon ordre. Elle parcourt simplement la table entière, ce qui ne se voit pas avec trente lignes et s'effondre avec trois mille. Un test automatisé vérifie le plan d'exécution pour cette raison précise — c'est une exigence non fonctionnelle transformée en test, et la seule manière d'empêcher une régression de performance de passer inaperçue pendant six mois.

Le second index est **partiel** : il ne couvre que les garages actifs et certifiés. Il est plus petit, donc plus rapide, et sert le tri par certification.

Le troisième est un index **GIN**, adapté aux tableaux : il accélère la question « ce garage offre-t-il tel service ? ».

LE RANG CALCULÉ EN SQL

```
ROW_NUMBER() OVER (ORDER BY
  CASE WHEN $sort = 'certified' THEN NOT certified END ASC,
  CASE WHEN $sort = 'rating' THEN score_note END DESC,
  distance_m ASC,
  id ASC
)
```

L'astuce repose sur une propriété du langage : dans une branche inactive, l'expression conditionnelle vaut la valeur nulle **pour toutes les lignes**. Elle ne discrimine donc rien, et le tri retombe sur la clé suivante.

Le résultat est qu'un seul plan de requête couvre les trois tris. On ne concatène pas de SQL selon le tri demandé — donc il n'existe aucune surface d'injection de ce côté.

La dernière clé, l'identifiant, garantit un ordre **total**. Sans elle, deux garages à distance égale pourraient s'échanger leurs rangs entre deux requêtes identiques, et les marqueurs se renumérotteraient sans raison visible.

LA NOTE BAYÉSIENNE

$$\text{score_note} = n/(n+20) \times \text{note} + 20/(n+20) \times 3,8$$

n	nombre d'avis du garage
20	seuil à partir duquel la note propre pèse pleinement
3,8	note moyenne de référence

Une moyenne simple laisserait un garage noté 5,0 sur deux avis complaisants devancer un garage noté 4,6 sur cent vingt-huit avis — exactement ce qu'un tri « mieux noté » ne doit pas faire.

La formule mélange la note observée avec une note de référence, en donnant d'autant plus de poids à la note observée que les avis sont nombreux. Avec deux avis, la note compte pour un onzième et la référence pour dix onzièmes. Avec cent vingt-huit avis, la note compte pour près de neuf dixièmes.

La note affichée sur la fiche du garage reste la note brute. La note pondérée sert à trier, pas à informer : elle serait incompréhensible pour un utilisateur.

5.11 LA GÉOMÉTRIE DU SERVICE ET LA PREUVE D'ARRIVÉE

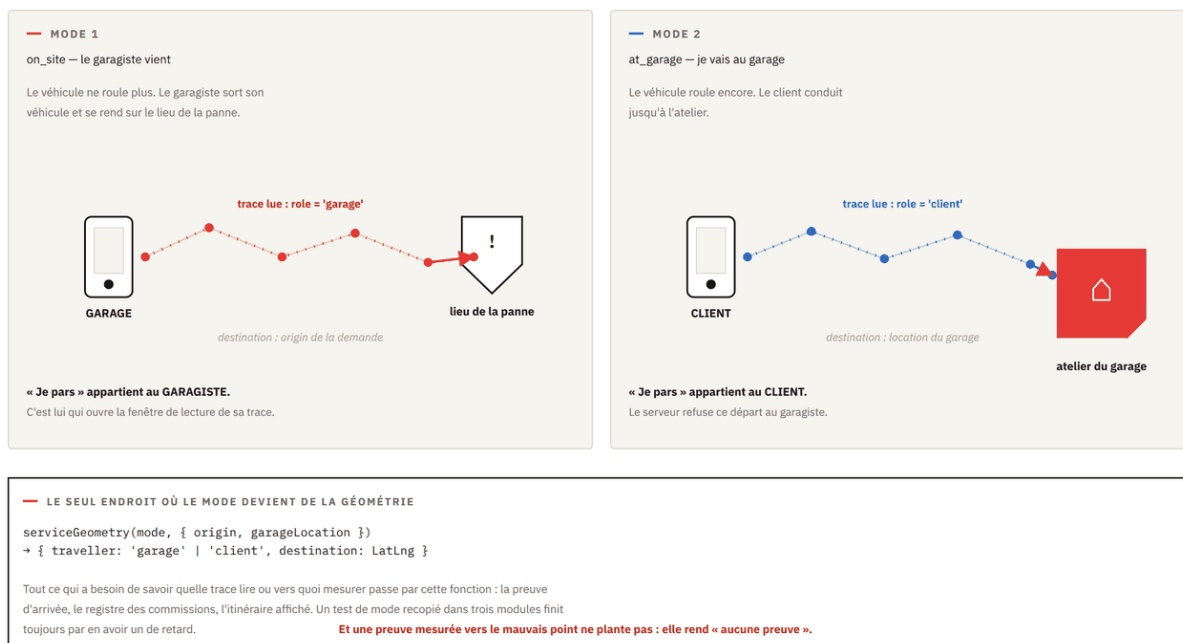


Figure 4 — Les deux modes de service, et la fonction unique qui les traduit en géométrie

Un dépannage se déroule dans deux sens. Ou bien le garagiste vient, ou bien le client conduit jusqu'à l'atelier. Toute la difficulté est que la facturation repose sur la trace de celui qui s'est déplacé — et que ce n'est pas la même personne selon le cas.

packages/shared/src/billing.ts

```
serviceGeometry('on_site', { origin, garageLocation })  
→ { traveller: 'garage', destination: origin }  
  
serviceGeometry('at_garage', { origin, garageLocation })  
→ { traveller: 'client', destination: garageLocation }
```

Tout ce qui a besoin de savoir quelle trace lire ou vers quoi mesurer passe par cette fonction : la preuve d'arrivée, le registre des commissions, l'itinéraire affiché. Un test de mode recopié dans trois modules finit toujours par en avoir un de retard — et une preuve mesurée vers le mauvais point ne plante pas, elle rend simplement « aucune preuve » sur des dépannages parfaitement réels.

LES TROIS SIGNAUX DE LA PREUVE

Seuils de `ARRIVAL_PROOF`

radiusMeters	150	distance sous laquelle on considère être arrivé
minDwellSeconds	240	durée minimale entre l'arrivée et la fin
minTravelMeters	200	distance minimale réellement parcourue

1. **`settled`** — le dernier point de la fenêtre est-il à moins de cent cinquante mètres de la destination ?
2. **`travelledMeters`** — le trajet a-t-il une longueur réelle, calculée par `pathLengthMeters` qui somme les haversines successives ?
3. **`dwellSeconds`** — la demande a-t-elle vécu assez longtemps après l'arrivée ?

Le premier signal tire parti d'une contrainte qui ressemblait à un défaut. Pour économiser les données, le téléphone n'émet un point qu'au-delà de quinze mètres parcourus : un véhicule à l'arrêt cesse donc complètement d'émettre. C'est en réalité le signal le plus fiable dont on dispose — une trace qui **s'arrête** au lieu de s'éloigner est exactement la signature d'une arrivée, là où un simple passage devant laisse des points qui continuent.

Tableau 10 — Les quatre niveaux rendus

Niveau	Ce qu'il établit	Facturable
none	Aucune trace n'établit un déplacement	Non
weak	Trajet trop court, ou intervention trop brève	Non
trail	La trace établit le déplacement	Oui
mutual	La trace l'établit et les deux parties en conviennent	Oui

Un point contre-intuitif à retenir : la double confirmation d'arrivée ne **crée** aucun droit à facturation, elle hisse d'un cran une preuve déjà acquise par la trace. Le raisonnement est simple — dès qu'une confirmation conditionnerait la facture, le débiteur aurait intérêt à la retenir.

La **fenêtre de lecture** est l'intervalle entre le départ déclaré et la fin connue de la demande. Tout relevé antérieur est ignoré. C'est pourquoi la transition de mise en route appartient exclusivement à celui qui se déplace, et pourquoi le serveur le vérifie : un garagiste qui poserait cet horodatage sur une demande où le client conduit démarrerait le chronomètre de quelqu'un d'autre — au mieux trop tôt, au pire après l'arrivée, ce qui effacerait la trace entière d'un trajet réel.

LE BACKEND, CHOIX PAR CHOIX

6.1 LE SQUELETTE D'EXPRESS

Le fichier `app.ts` assemble l'application, et son ordre est significatif : un intergiciel ne voit que ce que les précédents ont laissé passer.

L'ordre de montage, et ce qu'il implique

```
app.set('trust proxy', 1)    ← AVANT tout : sinon l'IP réelle est perdue
helmet()                    en-têtes de sécurité
cors({ origin: ... })        origines navigateur
express.json({ limit: '256kb' }) corps JSON, taille bornée
pinoHttp()                  journalisation (sauf en test)
GET /health                  sonde, avant l'authentification
/auth /garages /requests /me /driving /uploads
notFoundHandler              404 au format maison
errorHandler                 DERNIER : il capte ce qui a été levé
```

Le réglage `trust proxy` mérite une explication, parce qu'il est facile à oublier et coûteux. Derrière un hébergeur, l'adresse réelle du client se trouve dans un en-tête `X-Forwarded-For`, pas dans la connexion elle-même. Sans ce réglage, la limitation de débit voit une seule adresse — celle du proxy — et bloquerait donc tous les utilisateurs d'un coup au premier pic de trafic.

La limite de deux cent cinquante-six kilooctets sur le corps JSON est une protection : aucune requête légitime du produit n'approche cette taille, puisque les photographies ne passent pas par le serveur.

Le gestionnaire d'erreur est monté en dernier, et c'est une règle d'Express : il reçoit ce que les couches précédentes ont levé. Le monter plus haut le rendrait inerte.

6.2 LES TROIS BARRIÈRES DE VALIDATION

C'est le motif le plus important à comprendre du côté serveur. La même règle est appliquée à trois niveaux, et les trois sont utiles.

Tableau 11 — Les trois barrières, et ce que chacune protège

Barrière	Où	Ce qu'elle protège
Interface	l'écran désactive le bouton	l'utilisateur, d'une erreur
Contrat	le schéma partagé refuse la requête	le serveur, d'un client modifié ou d'une version ancienne
Base	la contrainte SQL refuse la ligne	nous-mêmes, d'un futur script d'import ou d'une reprise de données

La troisième est celle qu'on est tenté de sauter, et c'est la seule qui survive au temps. Une vérification écrite dans un service est vraie tant que tous les appelants passent par ce service ; elle cesse de l'être au troisième point d'entrée ajouté six mois plus tard par quelqu'un qui ignore la règle. Une contrainte inscrite dans le schéma reste vraie même quand le code se trompe.

C'est la discipline appliquée à toutes les règles qui touchent à l'argent ou à la preuve : la double confirmation d'arrivée, l'idempotence des crédits, l'unicité d'un garage par compte, l'interdiction du mode « je vais au garage » pour un véhicule immobilisé.

6.3 L'AUTHENTIFICATION

Le mécanisme, en cinq points

Jeton d'accès	JWT signé, durée courte (15 min par défaut)
Jeton de rafraîch.	chaîne opaque aléatoire, 30 jours
En base	seule l'EMPREINTE du jeton de rafraîchissement
Rotation	chaque usage produit une nouvelle paire et révoque l'ancienne
Mots de passe	bcrypt, 12 tours par défaut

Le jeton d'accès est un JWT : il porte l'identité et se vérifie par signature, sans interroger la base. C'est ce qui permet à chaque requête de coûter zéro lecture pour l'authentification.

Le jeton de rafraîchissement est l'inverse : il n'est pas un JWT mais une chaîne opaque, et il est stocké en base — mais seulement sous forme d'empreinte. Une fuite de la table ne donne donc aucune session utilisable, puisqu'on ne peut pas remonter de l'empreinte au jeton.

La rotation à chaque usage est ce qui limite les dégâts d'un vol. Un jeton dérobé et utilisé une fois est immédiatement révoqué ; le vol se détecte parce que le propriétaire légitime se retrouve déconnecté.

Le nombre de tours de bcrypt est réglable entre dix et quinze, avec douze par défaut. C'est un arbitrage entre coût de calcul et résistance : chaque tour double le temps de hachage. Douze tours représentent quelques centaines de millisecondes sur un serveur modeste — assez pour rendre une attaque par dictionnaire coûteuse, assez peu pour ne pas gêner une connexion.

6.4 LE CANAL TEMPS RÉEL, EN DÉTAIL

Les deux types de salles

request:<uuid>	une salle par demande d'assistance
user:<uuid>	une salle par compte, rejointe dès la connexion

La salle par demande est l'évidente : le client et le garagiste y échangent positions et durées d'arrivée.

La salle par compte règle un problème que les salles par demande ne peuvent pas résoudre. Au moment où un SOS est adressé à un garagiste, celui-ci **ne connaît pas encore cette demande** : il n'a donc rejoint aucune salle, et le seul canal capable de le prévenir est celui qui le désigne lui-même. C'est aussi ce qui tient plusieurs appareils du même garage d'accord : l'atelier accepte sur la tablette, le téléphone du dépanneur voit la file changer.

L'AUTHENTIFICATION DU SOCKET

Une connexion n'est pas authentifiée par le fait d'exister. À l'ouverture, le socket présente le jeton d'accès ; le serveur le vérifie et attache l'identité au socket. Puis, à chaque demande de rejoindre une salle, il vérifie **l'appartenance à la demande** — être authentifié ne suffit pas, il faut être partie à cette intervention précise.

LE REJEU APRÈS COUPURE

C'est le mécanisme qui rend le suivi robuste, et il repose entièrement sur une propriété du journal : `request_events` est en **ajout seul** et chaque événement porte un numéro de séquence unique au sein de sa demande.

1. Le client garde en mémoire le dernier numéro reçu.
2. Quand la connexion tombe, il bascule sur une interrogation périodique de la route d'état — la même route, exactement, que celle du premier chargement.
3. Au retour du réseau, il se réannonce en transmettant son dernier numéro.
4. Le serveur lit tous les événements postérieurs et les lui rejoue dans l'ordre.

L'effet secondaire est précieux : le mode dégradé et le premier chargement empruntent le même chemin de code. Il n'existe donc pas de branche rarement exécutée, donc rarement testée, qui ne fonctionnerait que le jour de la coupure.

6.5 LE FORMAT DES ERREURS

```
{ "error": { "code": "REQUEST_ALREADY_ACTIVE",  
            "message": "...",  
            "fields": { ... } } }
```

Le mobile traduit sur le **code**, jamais sur le message. La conséquence pratique est qu'on peut réécrire tous les messages du serveur sans casser un seul client, et qu'on peut afficher un texte différent selon la langue sans que le serveur ait à la connaître.

LA BASE DE DONNÉES : LIRE ET MODIFIER LE SCHÉMA

7.1 COMMENT LES MIGRATIONS FONCTIONNENT

Les migrations sont des fichiers SQL écrits à la main, numérotés, dans `apps/api/migrations/`. Le script `src/db/migrate.ts` les applique dans l'ordre et garde trace de ce qui a déjà tourné.

```
npm run db:migrate    applique ce qui manque
npm run db:reset      réinitialise puis réapplique tout
```

Tableau 12 — Les neuf migrations et ce que chacune apporte

Fichier	Apport
0001_init.sql	Le schéma complet : treize tables, index GIST et GIN, contraintes, déclencheurs, badges de référence
0002_opening_hours.sql	La fonction <code>garage_is_open</code> , pour que le filtre « ouverts » s'applique dans la requête de proximité
0003_request_vehicle_other.sql	Le véhicule et la panne de catégorie ouverte, avec les contraintes de libellé
0004_request_selected_at.sql	L'horodatage du choix du garage, pour que le compteur d'attente démarre au bon moment
0005_garage_single_owner.sql	Un index d'unicité : un garage par compte, même en écriture concurrente
0006_garage_verification.sql	Vérification distincte de la certification, et l'activité liée à la vérification
0007_request_declined.sql	Le refus comme événement de journal à part entière
0008_commission_ledger.sql	Le registre des commissions, en observation
0009_service_mode.sql	Les deux modes de service, la contrainte, et des commentaires en base

7.2 AJOUTER UNE MIGRATION

1. Créer `apps/api/migrations/00NN_ce_que_ca_fait.sql`, en incrémentant le numéro.
2. Écrire du SQL **idempotent** autant que possible : `IF NOT EXISTS`, `DROP CONSTRAINT IF EXISTS` avant `ADD CONSTRAINT`.
3. Documenter en tête du fichier **pourquoi**, pas seulement quoi. Les migrations existantes le font toutes, et c'est ce qui rend le schéma relisible un an plus tard.
4. Mettre à jour `src/db/types.ts` : Kysely ne devine pas le schéma, il le lit dans ce fichier.
5. Mettre à jour le contrat partagé si la forme exposée change.

6. Lancer `npm run db:migrate` sur la branche dev, puis `npm run typecheck`.

L'étape 4 est celle qu'on oublie, et son symptôme est déroutant : la colonne existe en base, la requête fonctionne, mais TypeScript prétend qu'elle n'existe pas. C'est normal — Kysely tire son typage de la déclaration, pas de la base.

Un usage de `COMMENT ON COLUMN` qui vaut d'être imité

La migration 0009 pose des commentaires SQL sur `garage_arrived_at` et `client_arrived_at`, parce que leurs noms datent du temps où un seul mode existait. En mode « je vais au garage », le garagiste n'« arrive » nulle part — il est déjà chez lui. Les renommer aurait coûté une contrainte SQL, un contrat partagé, quatre écrans et un registre, pour un gain de vocabulaire. La documentation vit donc dans la base, là où le prochain lecteur regardera.

7.3 LE SCHÉMA, EN VUE D'ENSEMBLE

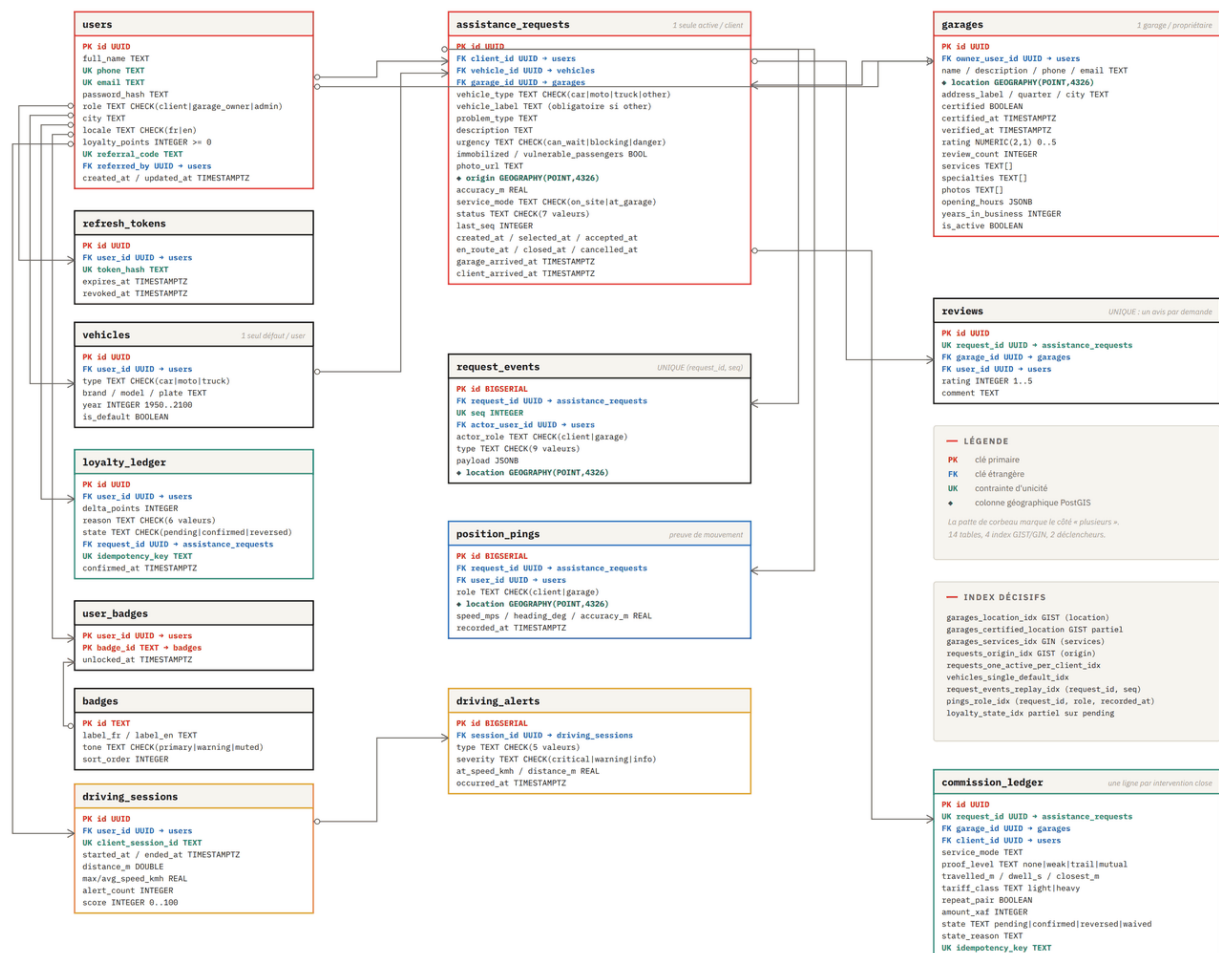


Figure 5 — Les quatorze tables, leurs clés, leurs contraintes et les index décisifs

LES TESTS

8.1 CE QUI EST COUVERT, ET PAR QUOI

Tableau 13 — Les trois suites

Espace de travail	Cas	Base requise
packages/shared	119	Non
apps/api	63	Partiellement
apps/mobile	145	Non

<code>npm test</code>	les trois espaces de travail
<code>npm test --workspace @geocras/shared</code>	une seule suite

8.2 LES TESTS GÉOSPATIAUX, ET POURQUOI ILS SONT IGNORÉS PAR DÉFAUT

Vingt et un tests de l'interface de programmation exigent une véritable instance PostgreSQL avec son extension. Ni une base en mémoire ni SQLite ne savent exécuter `ST_DWithin`, et tester la requête centrale du produit contre une imitation ne prouverait rien.

Ces tests sont donc **ignorés** tant qu'une base de test n'est pas fournie :

```
TEST_DATABASE_URL=postgresql://... npm test --workspace @geocras/api
```

Une branche dédiée, jamais dev, jamais main

La suite exécute `DROP SCHEMA public CASCADE` avant de reconstruire. Pointer cette variable sur votre branche de développement effacerait vos données de travail sans avertissement.

8.3 LE TEST QUI VÉRIFIE UN PLAN D'EXÉCUTION

Un test se distingue des autres : il ne vérifie pas un résultat mais la manière dont la base l'obtient. Il exige que la recherche de proximité passe par un parcours d'index géospatial et non par un parcours séquentiel.

C'est une exigence non fonctionnelle transformée en test, et c'est la seule manière d'empêcher une régression de performance de passer inaperçue. Un jour, quelqu'un réécrira cette requête « plus proprement » en mettant le calcul de distance dans le tri ; le résultat sera juste, les tests fonctionnels passeront, et la performance s'effondrera silencieusement. Celui-là échouera.

8.4 LES SCRIPTS D'AUDIT

```
npm run audit:arrivals    étalonne la preuve sur les données réelles  
npm run report:commissions relit le registre des commissions
```

Le premier relit toutes les interventions terminées et compte deux erreurs. Les **faux négatifs** sont les demandes closes — donc une rencontre a bien eu lieu — dont la preuve n'atteint pas le seuil facturable ; c'est l'erreur coûteuse, celle qui fait fuir les garages honnêtes. Les **faux positifs** sont les demandes annulées dont la preuve serait facturable ; c'est l'erreur qui casse la confiance dans l'autre sens.

Il ne modifie rien. Il faut le relancer maintenant que les deux modes de service coexistent : les seuils ont été calibrés sur des dépanneuses, et rien ne dit encore qu'ils conviennent à des clients au volant.

LIVRER ET METTRE À JOUR

9.1 LES PROFILS DE CONSTRUCTION

Tableau 14 — Les trois profils d'eas.json

Profil	Usage	Ce qu'il produit
development	Développement quotidien	Une version installable qui se connecte à Metro, avec les modules natifs
preview	Essais internes	Une version autonome, sans Metro, partageable
production	Publication	La version des magasins d'applications

```
eas build --profile development --platform android
eas build --profile preview --platform android
```

9.2 LES MISES À JOUR À DISTANCE

Le module `src/updates/useOtaUpdate.ts` et le fichier `apps/mobile/OTA.md` couvrent la mise à jour du code JavaScript sans repasser par un magasin d'applications.

Le principe et sa limite tiennent en deux phrases. Ce qui est écrit en JavaScript — écrans, logique, libellés, styles — se met à jour à distance. Ce qui est natif — un nouveau module, une permission ajoutée, une version du kit de développement — exige une nouvelle construction et une nouvelle installation.

La conséquence pratique : ajouter une bibliothèque à modules natifs n'est jamais une mise à jour à distance, quoi qu'en dise le fait que l'installation se fasse par npm.

LES PIÈGES, RASSEMBLÉS

Ce chapitre existe pour être relu quand quelque chose ne marche pas.

10.1 AU DÉMARRAGE

Tableau 15 — Symptôme, cause, remède

Symptôme	Cause	Remède
Cannot find module @geocras/shared	Le paquet partagé n'est pas compilé	<code>npm run build --workspace @geocras/shared</code>
Le serveur refuse de démarrer avec une liste de champs	Une variable d'environnement manque ou est trop courte	Lire la liste : elle nomme exactement les champs fautifs
Un changement du barème « ne prend pas »	Le paquet partagé n'a pas été recompilé	Lancer son mode veille dans un terminal à part
node_modules incohérent après un <code>npm install</code> en sous-dossier	Le lien symbolique du paquet interne est cassé	Supprimer le <code>node_modules</code> du sous-dossier, réinstaller depuis la racine

10.2 SUR LA CARTE

Tableau 16 — Symptôme, cause, remède

Symptôme	Cause	Remède
Onglet Carte vide, aucune erreur	L'application tourne dans Expo Go, qui n'embarque pas MapLibre	Construire un build de développement
Carte affichant « indisponible »	Clé MapTiler absente	Renseigner <code>EXPO_PUBLIC_MAPTILER_KEY</code>
Écran gris ou plantage au montage	Incompatibilité avec la nouvelle architecture	Passer <code>newArchEnabled</code> à <code>false</code> , reconstruire
La caméra ne bouge pas au cadrage	Un NaN dans la géométrie, ou un rectangle dégénéré	Vérifier <code>boundsOf</code> et <code>isDegenerate</code>
Une polyligne au milieu de l'océan	Inversion latitude/longitude	La convention d'OSRM ne doit pas sortir de <code>decodePolyline6</code>
Recherche lente en production	Le tri par distance précède le filtre	Remettre <code>ST_DWithin</code> avant le tri

10.3 SUR LE RÉSEAU ET LE SUIVI

Tableau 17 — Symptôme, cause, remède

Symptôme	Cause	Remède
----------	-------	--------

« Connexion impossible » sur toutes les requêtes	EXPO_PUBLIC_API_URL contient une chaîne vide	La laisser vide dans le fichier, l'IP de Metro est déduite
Le point de l'utilisateur téléporte	Filtre de position contourné	Tout relevé doit passer par <code>PositionFilter.accept</code>
Durée d'arrivée qui saute	Vitesse brute au lieu de la vitesse lissée	Utiliser <code>smoothSpeed</code>
Aucun relevé pendant plusieurs minutes	Le véhicule est à l'arrêt : le seuil de quinze mètres suspend l'émission	Comportement normal, et c'est le signal de l'arrivée
Le garagiste ne voit pas un nouveau SOS	Il n'a pas rejoint sa salle de compte	Vérifier l'authentification du socket à la connexion

10.4 SUR LA FIDÉLITÉ ET LES COMMISSIONS

Tableau 18 — Symptôme, cause, remède

Symptôme	Cause	Remède
Une demande refuse de se clore	Une seule confirmation d'arrivée sur les deux	C'est la contrainte <code>closed_requires_both_arrivals</code> qui parle : elle a raison
Un crédit compté deux fois	Clé d'idempotence non employée	Passer par <code>ledgerIdempotencyKey</code>
Preuve none sur une intervention réelle	Trace lue pour le mauvais voyageur	Tout passe par <code>serviceGeometry</code> , jamais par un test de mode recopié
Le départ est refusé au garagiste	La demande est en mode « je vais au garage »	Comportement voulu : le départ appartient au voyageur

COMMANDES DE RÉFÉRENCE

Tableau 19 — Tout ce qu'on tape, réuni

Commande	Effet
<code>npm install</code>	Installe les trois paquets, depuis la racine uniquement
<code>npm run build --workspace @geocras/shared</code>	Compile le paquet partagé — obligatoire avant tout
<code>npm run dev --workspace @geocras/shared</code>	Recompile le paquet partagé à chaque sauvegarde
<code>npm run api</code>	Lance le serveur, rechargement à chaud, port 3000
<code>npm run mobile</code>	Lance Metro pour l'application
<code>npm run android / npm run ios</code>	Construit et installe en local
<code>npm run db:migrate</code>	Applique les migrations manquantes
<code>npm run db:reset</code>	Réinitialise puis réapplique tout
<code>npm run db:seed</code>	Trente garages à Yaoundé, plus le contrôle du plan d'index
<code>npm run db:seed:ebolowa</code>	Trois garages à Ebolowa, pour les essais de terrain
<code>npm run db:seed:garagiste</code>	Un compte propriétaire de garage
<code>npm run typecheck</code>	Vérifie les types des trois espaces de travail
<code>npm test</code>	Exécute les trois suites de tests
<code>npm run audit:arrivals</code>	Étalonne la preuve d'arrivée sur les données réelles
<code>npm run report:commissions</code>	Relit le registre des commissions
<code>eas build --profile development --platform android</code>	Version installable de développement
<code>curl localhost:3000/health</code>	Sonde du serveur

11.1 LES CONSTANTES À CONNAÎTRE DE MÉMOIRE

Tableau 20 — Les valeurs qui gouvernent le comportement

Constante	Valeur	Où
Cadence d'émission	4 s	<code>EMISSION.throttleMs</code>
Seuil de déplacement	15 m	<code>EMISSION.minMoveMeters</code>
Repli en mode dégradé	15 s	<code>EMISSION.pollIntervalMs</code>
Donnée signalée périmée	20 s	<code>EMISSION.staleAfterMs</code>
Seuil de reconnaissance mutuelle	120 m	<code>PROXIMITY</code>
Lissage de la vitesse	$\alpha = 0,3$	<code>SPEED_SMOOTHING_ALPHA</code>

Vitesse implausible	150 km/h	MAX_PLAUSIBLE_SPEED_MPS
Précision maximale acceptée	200 m	PositionFilter.MAX_ACCURACY_M
Facteur de détour	1,35	ETA_CALIBRATION
Vitesse urbaine	22 km/h	ETA_CALIBRATION
Rayon d'arrivée	150 m	ARRIVAL_PROOF.radiusMeters
Durée minimale sur place	240 s	ARRIVAL_PROOF.minDwellSeconds
Distance minimale parcourue	200 m	ARRIVAL_PROOF.minTravelMeters
Seuil de la note bayésienne	20 avis	BAYES.minReviews
Note de référence	3,8	BAYES.priorRating
Points par intervention	50	POINTS
Différé avant confirmation	24 h	ANTI_FRAUD.pendingPeriodHours
Tarif léger / lourd	500 / 1 500 XAF	TARIFF_XAF
Remise paire récurrente	moitié	REPEAT_PAIR_FACTOR

11.2 OÙ REGARDER EN PREMIER

Tableau 21 — La question, et le fichier qui y répond

Question	Fichier
« Quelle est la forme de cette réponse d'API ? »	packages/shared/src/contracts/
« Combien de points pour telle action ? »	packages/shared/src/loyalty.ts
« Quelles pannes pour tel véhicule ? »	packages/shared/src/taxonomy.ts
« Comment la preuve d'arrivée est-elle jugée ? »	packages/shared/src/billing.ts
« Comment la distance est-elle calculée ? »	packages/shared/src/geo.ts
« Quelles sont les tables et leurs contraintes ? »	apps/api/migrations/0001_init.sql
« Comment la recherche de proximité est-elle écrite ? »	apps/api/src/modules/garages/garages.repo.ts
« Qui a le droit de faire quoi sur une demande ? »	apps/api/src/modules/requests/requests.service.ts
« Comment les salles temps réel fonctionnent-elles ? »	apps/api/src/realtime/io.ts
« Quelles variables d'environnement existent ? »	apps/api/src/config/env.ts
« Quelles couleurs, quelles polices ? »	apps/mobile/src/theme/tokens.ts et typography.ts
« Comment la carte est-elle stylée ? »	apps/mobile/src/map/style.ts
« Comment les positions sont-elles filtrées ? »	apps/mobile/src/location/filter.ts
« Quels écrans existent ? »	apps/mobile/app/